



UNIVERSIDAD CATÓLICA DEL NORTE
FACULTAD DE INGENIERÍA Y CIENCIAS
GEOLÓGICAS
DEPARTAMENTO DE INGENIERÍA DE SISTEMAS Y
COMPUTACIÓN

Extracción Iterativa de Conos Invertidos Positivos
en un Modelo de Bloques 3D

Integrantes:

Camilo Jofré Tapia

Jhoan Montero Huarachi

Sebastián Hernández Lara

Profesor:

Aldo Quelopana Retamal

Diseño y Análisis de Algoritmos

28/05/2026

Ingeniería Civil en Computación e Informática

Introducción

La planificación minera a cielo abierto constituye uno de los problemas de optimización combinatoria más complejos y relevantes de la industria extractiva. En este contexto, la determinación del orden óptimo de extracción de bloques no solo debe considerar su valor económico individual, sino también las restricciones geométricas y de estabilidad que condicionan qué bloques pueden removerse en cada momento del proceso productivo. Estas restricciones generan relaciones de precedencia entre bloques, cuya correcta modelación y explotación algorítmica es fundamental para maximizar el valor del yacimiento.

El presente trabajo aborda, desde la perspectiva del diseño y análisis de algoritmos, el problema de la extracción iterativa de conos invertidos positivos sobre un modelo de bloques tridimensional. Un cono invertido representa el conjunto mínimo de bloques que deben removerse para que un bloque objetivo pueda ser extraído de manera factible, respetando estrictamente las relaciones de precedencia entre bloques. El interés particular de este tema radica en que la extracción de conos positivos, aquellos cuya suma de valores económicos es estrictamente mayor que cero, debe realizarse de forma iterativa, actualizando el estado del yacimiento en cada paso, y deteniéndose cuando ya no existan conos positivos disponibles.

Este problema está directamente relacionado con el problema de la mina a cielo abierto, ampliamente estudiado en la investigación de operaciones y optimización combinatoria. Sin embargo, el enfoque adoptado en este trabajo no apunta a resolver el problema completo de planificación minera, sino a estudiar el comportamiento algorítmico de estrategias iterativas de extracción de conos, analizando su complejidad teórica y evaluando empíricamente su desempeño sobre instancias generadas a partir de modelos de bloques reales o sintéticos.

La presente entrega corresponde a la primera etapa del proyecto. En ella se establece una definición precisa del problema abordado, se revisa la literatura relacionada, se propone una formulación algorítmica y se diseña una línea base experimental que servirá como punto de comparación para las etapas posteriores.

2. Definición del problema

2.1 Descripción del problema

Se dispone de un modelo de bloques tridimensionales que representa un yacimiento minero. Cada bloque está identificado por sus coordenadas enteras (x, y, z) en el espacio tridimensional y tiene asociado un valor económico $v(x, y, z) \in \mathbb{R}$, que puede ser positivo (mineral de interés), negativo (costo de extracción sin mineral) o cero. El objetivo es extraer iterativamente conos invertidos positivos, maximizando el valor económico acumulado obtenido.

2.2 Notación

Se utilizará la siguiente notación a lo largo del trabajo:

- B : conjunto de todos los bloques del yacimiento.
- $b = (x, y, z)$: bloque en la posición x, y, z donde $x, y, z \in \mathbb{Z}^+$.
- $v(b)$: valor económico asociado al bloque b .
- $E \subseteq B$: conjunto de bloques extraídos, donde cada bloque se contabiliza una única vez.
- V_{Total} : El valor económico total extraído, calculado como la suma de $v(b)$ para todo b en E .
- $R \subseteq B$: conjunto de bloques ya extraídos (inicialmente vacío).
- $\text{pred}(b)$: conjunto de predecesores directos del bloque b (bloques cuya extracción es condición necesaria para extraer b).
- $C(b)$: cono invertido del bloque b dado el estado actual del yacimiento.
- $V(C)$: valor económico total del cono C , definido como la suma de $v(b')$ para todo b' en C .

2.3 Entradas del Problema

El algoritmo recibe como entrada:

- Un modelo de bloques 3D representado como un conjunto $B = \{(x_i, y_i, z_i, v_i) \mid i = 1, \dots, n\}$, donde n es el número total de bloques.
- La regla de precedencia: el bloque (x, y, z) puede extraerse solo si los 9 bloques ubicados en las posiciones $(x+i, y+j, z+1)$ para $i, j \in \{-1, 0, 1\}$ existen en el modelo y han sido previamente extraídos.

2.4 Salidas del Problema

El algoritmo debe producir:

- El conjunto total de bloques extraídos $E \subseteq B$, donde cada bloque se contabiliza una única vez.
- El valor económico total extraído: $V_{total} = \sum v(b)$ para todo b en E .
- El registro de bloques no extraíbles detectados durante el proceso (ver 2.5), junto con la causa (predecesor superior faltante).

2.5 Restricciones y Supuestos

El modelo considera las siguientes restricciones y supuestos:

- Bloque de superficie: un bloque $b = (x, y, z)$ es de superficie si no existe ningún bloque en la posición $(x, y, z+1)$ en el modelo. Los bloques de superficie pueden extraerse directamente sin ninguna condición previa.
- Bloque no superficial extraíble: un bloque no superficial cuyos 9 predecesores en $(x+i, y+j, z+1)$, $i, j \in \{-1, 0, 1\}$, existen en el modelo. Este bloque puede extraerse una vez que esos 9 predecesores ya fueron extraídos.
- Bloque no superficial y no extraíble: un bloque no superficial al que le falta al menos uno de los 9 predecesores requeridos (es decir, esa posición no existe en el modelo). Este bloque no puede extraerse bajo ninguna circunstancia, ya que no se permiten bloques ficticios que sustituyan al predecesor ausente.
- Regla de precedencia estricta: un bloque no superficial solo puede extraerse si los 9 bloques superiores existen en el modelo y todos han sido previamente extraídos. Una posición inexistente no se considera ya extraída debido a que su ausencia hace que el bloque objetivo, y por extensión cualquier cono que dependa de él, no sea extraíble.
- Extracción atómica de conos: cada iteración extrae exactamente un cono invertido positivo completo.
- Contabilización única: si un bloque ha formado parte de un cono extraído en una iteración anterior, no se vuelve a contabilizar ni a extraer.

2.6 Objetivo del Algoritmo

Dado el modelo de bloques B , construir las relaciones de precedencia entre bloques, e identificar y extraer iterativamente conos invertidos positivos, actualizando el estado del yacimiento tras cada extracción, hasta que no existan más conos invertidos con valor estrictamente positivo. El valor económico total extraído debe calcularse sumando el valor de cada bloque removido exactamente una vez.

2.7 Condición de Término

El proceso iterativo se detiene cuando, para todo bloque $b \in B \setminus R$ (bloques aún no extraídos), se cumple que $V(C(b)) \leq 0$ o no es extraíble. Es decir, el yacimiento remanente está compuesto únicamente por bloques cuyos conos invertidos tienen valor económico no positivo, o que no pueden extraerse por falta de predecesores. En el caso de que aparezcan conos con valor exactamente igual a cero durante la experimentación, el grupo reportará estos casos y explicará el criterio adoptado para tratarlos.

2.8 Ejemplo Conceptual

Para ilustrar el problema, considérese un modelo simplificado de bloques con tres niveles ($z = 0$, $z = 1$ y $z = 2$).

Sea el bloque objetivo $b_0 = (3, 3, 0)$, con $v(b_0) = -4$. Como b_0 no es de superficie, sus predecesores directos son los 9 bloques en $z = 1$: $(2,2,1)$, $(2,3,1)$, $(2,4,1)$, $(3,2,1)$, $(3,3,1)$, $(3,4,1)$, $(4,2,1)$, $(4,3,1)$ y $(4,4,1)$.

De estos 9 bloques, supongamos que $(3,3,1)$ tampoco es de superficie (existe un bloque en $(3,3,2)$), por lo que a su vez requiere sus propios 9 predecesores en $z = 2$: $(2,2,2)$, $(2,3,2)$, $(2,4,2)$, $(3,2,2)$, $(3,3,2)$, $(3,4,2)$, $(4,2,2)$, $(4,3,2)$ y $(4,4,2)$. Aquí es donde aparece la recursión: para poder extraer $(3,3,1)$ —y por lo tanto b_0 — primero deben extraerse estos 9 bloques de $z = 2$.

Los otros 8 bloques de $z = 1$, en cambio, sí son de superficie (no existe bloque alguno en su posición $(x, y, 2)$), por lo que pueden extraerse directamente sin condición previa.

El cono invertido completo de b_0 es entonces: $C(b_0) = \{b_0\} \cup \{8 \text{ bloques de superficie en } z=1\} \cup \{(3,3,1)\} \cup \{9 \text{ bloques en } z=2\}$, un total de 19 bloques. Si la suma de sus valores es mayor que cero, el cono es positivo y debe extraerse en alguna iteración. Si alguno de los 9 bloques de $z = 2$ no existiera en el modelo, $(3,3,1)$, y por lo tanto b_0 , no sería extraíble, y el cono no se extraería en absoluto, independientemente de su valor.

3. Revisión Bibliográfica

A continuación se menciona cada referencia utilizada y cómo éstas influyeron en la manera en que se abordó el problema.

- Lerchs y Grossmann (1965) formalizaron el problema del diseño óptimo de pit como un problema sobre grafos dirigidos con relaciones de precedencia, siendo la base histórica de todo el área. Se utiliza para justificar que este trabajo modele el yacimiento como un grafo de precedencias entre bloques en lugar de una estructura ad-hoc.
- Picard (1976) demostró que el problema de diseño de pit óptimo es equivalente al problema de cierre máximo ("maximal closure") en un grafo. Esta referencia sustenta por qué la extracción de un cono invertido completo es la unidad correcta de decisión en lugar de utilizar bloques individuales: un cono es, en esencia, un cierre parcial del grafo de precedencias.
- Ares et al. (2022) describen y evalúan el método del cono flotante ("floating cone") como heurística práctica para el diseño de pit. Esta es la base directa de la línea base implementada en este trabajo, al ser la estrategia de reconstruir y evaluar conos iterativamente una variante simplificada de floating cone.
- Dowd y Onur (1993) discuten el diseño óptimo de pit desde una perspectiva de optimización aplicada a la industria minera. Se utiliza para contextualizar por qué la extracción iterativa de conos positivos es una simplificación razonable —aunque no óptima globalmente— del problema real de planificación de pit.
- Hochbaum (2008) propone el algoritmo pseudoflow como método eficiente para resolver el problema de cierre máximo, superando en la práctica a métodos de flujo máximo clásicos. Esta es una alternativa más eficiente que la línea base de fuerza bruta, y se considera como alternativa para mejorar el algoritmo en futuras entregas.
- Turan y Onur (2023) presentan una versión mejorada del algoritmo de cono flotante que corrige algunas de sus limitaciones de optimización conocidas. Este se considera como una referencia concreta de mejora incremental sobre el mismo tipo de algoritmos que se está utilizando como línea base, y como referencia para realizar mejoras en el diseño del algoritmo para futuras entregas.

En conjunto, estas fuentes confirman que el enfoque adoptado (fuerza bruta con recálculo de conos) corresponde a una variante del floating cone algorithm, y que existen métodos con mejor complejidad (pseudoflow, cierre máximo) que se pueden explorar como alternativas para próximas entregas.

4. Línea base para el problema

4.1 Descripción de la estrategia

La línea base se implementa como una versión simple del Algoritmo del Cono Flotante. En cada ciclo, el sistema reconstruye completamente el cono invertido para cada bloque candidato extraíble, evalúa su valor y selecciona para la extracción el primer cono positivo que encuentra al recorrer el modelo. Este proceso se repite hasta que no quede ningún cono positivo entre los bloques extraíbles. Si bien esta estrategia es correcta, ya que solo extrae los bloques que constituyen un cono positivo, respeta las precedencias y descarta explícitamente los conos que dependen de un bloque no extraíble, su principal limitación es la ineficiencia. El costo de recalcular los conos completos en cada ciclo, sin reutilizar el trabajo previo, puede ser alto en modelos extensos.

4.2 Pseudocódigo

Algoritmo: Línea Base – Extracción Iterativa de Conos (Fuerza Bruta)

```
DEF construir_precedencias(B):
    prec ← mapa vacío
    For each bloque b = (x, y, z) in B:
        prec[b] ← {}
        es_superficie ← (x, y, z+1) ∉ B
        If es_superficie:
            CONTINUE
        faltantes ← 0
        For i in {-1, 0, 1}:
            For j in {-1, 0, 1}:
                p ← (x+i, y+j, z+1)
                If p exists in B:
                    prec[b].add(p)
                Else:
                    faltantes ← faltantes + 1
        If faltantes > 0:
            marcar b como INEXTRAIBLE
    RETURN prec

DEF calcular_cono(b, R, prec, inextraibles):
    If b in inextraibles:
        RETURN INEXTRAIBLE
    cono ← {}
    pila ← [b]
    While pila:
        actual ← pila.pop()
        If actual in R or actual in cono:
            continue
        If actual in inextraibles:
            RETURN INEXTRAIBLE
        cono.add(actual)
        For each p in prec[actual]:
            If p ∉ R and p ∉ cono:
                pila.push(p)
    RETURN cono
```

```

ALGORITMO línea_base(B):
    prec, inextraibles = construir_precedencias(B)
    R = ∅, E = ∅, Vtotal = 0
    conos_cero = []
    cambio = TRUE
    While cambio:
        cambio = FALSE
        For each b in orden si b ∉ R:
            cono = calcular_cono(b, R, prec, inextraibles)
            If cono == INEXTRAIBLE:
                continue
            valor = SUMA v(b') for b' in cono
            If valor == 0:
                conos_cero.append((b, cono))
            If valor > 0:
                R = R ∪ cono
                E = E ∪ cono
                Vtotal = Vtotal + valor
                cambio = TRUE
                BREAK
    RETURN E, Vtotal, conos_cero

```

4.3 Análisis de complejidad

La complejidad se analiza considerando $n = |B|$, el número total de bloques. En el escenario menos favorable:

- La construcción de precedencias es $O(n)$, ya que cada bloque tiene un máximo de nueve predecesores directos.
- La función `calcular_cono` puede recorrer todos los bloques no extraídos, lo que implica una complejidad $O(n)$.
- Dentro del ciclo principal (while), el bucle for recorre hasta n bloques y llama a `calcular_cono` en cada uno, lo que suma $O(n^2)$ por iteración.
- El número de iteraciones del ciclo while es, a lo sumo, n , dado que cada iteración exitosa garantiza la extracción de al menos un bloque.

La complejidad total del algoritmo de línea base es $O(n^3)$ en el peor caso. Este costo se incrementa con rapidez a medida que aumenta el tamaño del modelo.

4.4 Validez y criterio de término

La línea base garantiza soluciones válidas por las siguientes razones:

- La función `calcular_cono` genera el conjunto mínimo de bloques no extraídos que requiere el bloque objetivo mediante una búsqueda exhaustiva tipo DFS con pila, y ahora retorna explícitamente que este no es extraíble cuando el cono depende de un predecesor faltante
- La extracción solo se realiza si el cono tiene un valor económico estrictamente positivo y este se considera extraíble.
- Una vez extraídos, los bloques se registran en R y se excluyen de futuros procesamientos.
- El algoritmo está garantizado para terminar. En cada iteración exitosa, el conjunto de bloques remanentes extraíbles disminuye al menos en uno, y como el conjunto B es finito, el proceso debe finalizar.

4.5 Política de selección

La política de selección adoptada por la línea base es la más sencilla: se elige el primer cono positivo que aparece durante el recorrido del modelo siguiendo el orden fijo (z descendente, (x,y)). Aunque esta política facilita la implementación y la verificación de la corrección algorítmica, no está diseñada para optimizar el orden de extracción. Las futuras entregas explorarán políticas alternativas, tales como la selección por mayor valor total o por la mejor razón valor/tamaño.

5. Diseño Experimental Preliminar

5.1 Instancias de prueba

Instancia	Dimensiones	Origen	Propósito
I1	3×3×2	Sintética, semilla fija	Validación de correctitud y caso de bloques no extraíbles
I2	5×5×5	Sintética, semilla fija	Medición de tiempo/memoria a escala pequeña
I3	10×10×10	Sintética, semilla fija	Medición de tiempo/memoria a escala media
I4	Según entrega del profesor	Real / provista por el curso	Validación final y comparación con algoritmo mejorado

5.2 Métricas a registrar

- Tiempo de ejecución total (ms).
- Memoria máxima utilizada (MB).
- Valor económico total extraído (V_{total}).
- Número de conos extraídos.
- Número total de bloques extraídos.
- Número de bloques marcados como no extraíbles.
- Número de conos de valor cero detectados y reportados.

5.3 Protocolo experimental

- Cada instancia sintética se ejecutará 5 veces, reportando promedio y desviación estándar de tiempo y memoria.
- Las instancias aleatorias se generarán con semillas fijas (42, 123, 2026) para garantizar la reproducibilidad.
- La instancia entregada por el profesor se ejecutará una vez, dado que sus valores no son aleatorios.
- Hardware de referencia: se documentará CPU, RAM y sistema operativo de la máquina donde se ejecuten los experimentos, para contextualizar los tiempos reportados.

5.4 Protocolo experimental

Antes de correr experimentos de desempeño, se validará la correctitud del algoritmo con casos pequeños de solución conocida, calculada a mano:

- Caso 1 — Cono positivo simple: modelo de 2 niveles donde el cono completo tiene valor positivo conocido; se verifica que el algoritmo extraiga exactamente ese conjunto de bloques.
- Caso 2 — Bloque no extraíble (caso de borde): modelo donde al bloque objetivo le falta uno de sus 9 predecesores superiores (posición fuera del modelo). Se verifica que el algoritmo no lo extraiga en ninguna iteración y lo reporte como no extraíble.
- Caso 3 — Cono de valor cero: modelo diseñado para que la suma de un cono sea exactamente 0. Se verifica que el algoritmo lo detecte, lo reporte en la lista de conos_cero, y no lo extraiga.
- Caso 4 — Recursión multinivel: modelo de 3 niveles para verificar que la recursión hacia arriba se calcule correctamente.

6. Plan de Trabajo del Grupo

6.1 Distribución de Tareas

Con el fin de que el desarrollo del proyecto se realice de manera estructurada, se asignaron roles a cada integrante del grupo con responsabilidades específicas, de manera que las tareas se puedan dividir de manera adecuada, estos roles son:

- **Investigación:** Dedicado a la investigación sobre posibles mejoras aplicables al algoritmo utilizado como base o alternativas a este, principalmente relacionado a la búsqueda de fuentes.

Integrante asignado: Jhoan Montero

- **Implementación:** Dedicado a la implementación de los algoritmos encontrados siguiendo los estándares de programación correspondientes.

Integrante asignado: Camilo Jofré

- **Encargado de SQA:** Dedicado a realizar pruebas para asegurar el correcto funcionamiento del algoritmo, documentación y de encontrar posibles puntos de mejora en las implementaciones.

Integrante asignado: Sebastián Hernández

6.2 Cronograma para las siguientes entregas

Para la Entrega 2 (fecha límite: 19 de junio):

- Semana del 2 de junio: Implementar línea base en C++ y comenzar diseño del algoritmo mejorado.
- Semana del 9 de junio: Empezar la implementación del algoritmo mejorado.
- Semana del 16 de junio: Completar análisis de complejidad, resultados preliminares y redactar informe.

Para la Entrega 3 (fecha límite: 30 de junio):

- Semana del 23 de junio: Refinar algoritmos, validaciones finales de su funcionamiento y redacción del informe final.
- Semana del 28 de junio: Revisión final del código, instrucciones de ejecución y preparación de la presentación oral.

Junto a la anterior, se utilizará Trello para el seguimiento de las actividades a realizar, como lo son reuniones, etc. El enlace a este se puede ver cómo el anexo 1.

7. Conclusión

En esta entrega se logró definir el marco general para el desarrollo del proyecto sobre extracción iterativa de conos invertidos positivos en modelos de bloques tridimensionales. Primero, se formuló el problema utilizando una notación precisa, definiendo las entradas del modelo (modelo de bloques B y regla de precedencia 1:9), las salidas esperadas (conjunto de bloques extraídos E y valor total V_{total}), además de las restricciones y la condición de término del algoritmo. También se definió formalmente el concepto de cono invertido como el conjunto mínimo de bloques necesarios para que un bloque objetivo pueda extraerse respetando las precedencias, incluyendo su formulación recursiva y el criterio de positividad.

Además, se realizó una revisión bibliográfica de seis referencias académicas relacionadas con el diseño de tajos a cielo abierto. Esto permitió contextualizar el problema dentro del marco teórico de clausura máxima en grafos y reconocer que el enfoque iterativo utilizado corresponde a una variante del Floating Cone Algorithm. A partir de la literatura revisada, se identificó que este tipo de algoritmos presenta limitaciones respecto a su optimización, lo que justifica la búsqueda de mejoras en las próximas etapas del proyecto.

Como línea base, se diseñó e implementó conceptualmente una estrategia basada en la reconstrucción exhaustiva de conos en cada iteración. Se comprobó que este enfoque es correcto, ya que respeta las relaciones de precedencia, extrae únicamente conos con valor positivo y garantiza la terminación del proceso. Sin embargo, también se identificó como principal dificultad su alta complejidad computacional en el peor caso, de complejidad $O(n^3)$, lo que podría volverlo poco eficiente para modelos de gran tamaño. La política de selección utilizada —extraer el primer cono positivo encontrado— se adoptó como una solución simple que servirá de referencia para comparar métodos más avanzados.

Para las siguientes entregas, el trabajo se enfocará en reducir el costo de reconstrucción de conos mediante técnicas que eviten cálculos repetidos, además de desarrollar políticas de selección que permitan obtener mejores resultados finales. Finalmente, se proyecta implementar y validar experimentalmente el algoritmo en C++ utilizando el modelo de bloques entregado por el profesor.

8. Bibliografía

Ares, G., Castañón Fernández, C., Diego Álvarez, I., Arias, D., & Bolaño Benítez, A. (2022). Open pit optimization using the floating cone method: A new algorithm. *Minerals*, 12(4), 495. <https://doi.org/10.3390/min12040495>

Dowd, P. A., & Onur, A. H. (1993). Open-pit optimization – Part 1: Optimal open-pit design. *Transactions of the Institution of Mining and Metallurgy, Section A: Mining Industry*, 102, A95–A104.
https://www.researchgate.net/publication/291116522_Open-pit_optimization_-_Part_1_Optimal_open-pit_design

Hochbaum, D. S. (2008). The pseudoflow algorithm: A new algorithm for the maximum-flow problem. *Operations Research*, 56(4), 992–1009. <https://doi.org/10.1287/opre.1080.0524>

Lerchs, H., & Grossmann, I. F. (1965). Optimum design of open-pit mines. *Transactions of the Canadian Institute of Mining and Metallurgy*, 68, 17–24.

Picard, J.-C. (1976). Maximal closure of a graph and applications to combinatorial problems. *Management Science*, 22(11), 1268–1272. <https://doi.org/10.1287/mnsc.22.11.1268>

Turan, G., & Onur, A. H. (2023). Optimization of open-pit mine design and production planning with an improved floating cone algorithm. *Optimization and Engineering*, 24, 1157–1181. <https://doi.org/10.1007/s11081-022-09725-4>

9. Anexos

ANEXO A: TRELLO

<https://trello.com/invite/b/6a18f6da8cc1f5a22b6d1702/ATTI82cbe83cf6df2e71bb149f55154368ad022931BA/proyecto-diseno-y-analisis-de-algoritmos>

